Movie Title Autocomplete

Comparing a Trie and a Ternary Search Tree

Movie Search | Group 123 | COP3530 Project 2b | Report draft

Team: Aidan Quach (aidanq06); Hana Elbatouty (hana4324); Krish Gupta (KrishGupta25).

Repository: https://github.com/aidanq06/movie-title-autocomplete

Video: https://youtu.be/XtHltlB4EbI

Problem and motivation

Movie title autocomplete helps users find a film when they remember only the beginning of its name. A full scan repeatedly checks the dataset, while a prefix index groups titles so matching records can be reached through shared characters. Our project investigates how two manually implemented structures, a Trie and a Ternary Search Tree (TST), solve the same prefix-search problem on the same 150,000 records [1].

Both structures must return the same movies in the same order. Their differences should concern construction cost, query latency, node visits, and estimated index storage. This gives us a practical way to compare representation choices rather than simply comparing asymptotic labels. The project also requires reliable input handling and reproducible experiments, because inconsistent data selection or result ordering would invalidate the comparison.

Implemented features

The C++17 terminal application offers Trie search, TST search, side-by-side comparison, a performance benchmark, dataset/index statistics, and CSV export. Searches return up to ten records with original titles, release years, and IMDb IDs. Missing years display as Unknown year. Empty normalized prefixes and nonexistent prefixes return no matches. Both indexes support rebuilding and keep duplicate-title movies as separate records.

Data and selection

We use IMDb's official non-commercial title.basics.tsv.gz dataset, decompressed locally into a UTF-8 tab-separated file [2]. The loader streams one line at a time and maps required fields by header name. It selects the first 150,000 eligible rows in file order: titleType is movie, isAdult is 0, the IMDb ID and primary title are present, and normalization produces a nonempty key. No title deduplication occurs. Missing or invalid years become -1; malformed row widths are counted and skipped. Missing files and invalid headers raise errors.

The reported Windows experiment used a snapshot downloaded on November 4, 2026, at 22:57:51 UTC; its full SHA-256 digest is recorded in the analysis notes [3]. Selecting 150,000 movies scanned 270,772 source rows. The selection exceeds the required 100,000 data points, but its original-file ordering favors older films. The large source dataset is excluded from Git; a small synthetic fixture is committed for reproducible tests.

Manual structures and shared behavior

The Trie stores all nodes in a vector, with root index 0. Each node owns a sorted vector of character/child-index pairs and a vector of terminal movie IDs. Insertion follows characters and creates missing nodes. Search follows the normalized prefix and then uses iterative depth-first traversal, emitting terminal IDs before ordered children. It stops when the requested result limit is reached. This sparse representation avoids a large fixed child-pointer array at every node.

The TST also stores nodes in a vector. Each node contains one character, left/middle/right child indices, and terminal movie IDs. Left and right links represent alternative characters at the same title position; a middle link advances to the next character. After locating a prefix endpoint, search emits its terminal records and traverses only its middle subtree in left/node/middle/right order. The endpoint's own left/right siblings do not share the prefix. Traversal is iterative in both structures, avoiding recursion overflow on long titles.

Both are implemented from scratch behind PrefixIndex, which exposes build, search, stats, and name [1]. A MovieId is an index into one shared, immutable movie vector. Standard containers support node storage; no library Trie or TST replaces the implementations.

Normalization and auxiliary algorithms

Titles and raw queries use the same normalizer: lowercase ASCII uppercase letters, retain letters and digits, replace unsupported bytes with spaces, collapse spaces, and trim. Spider-Man: Homecoming becomes spider man homecoming. This is not Unicode transliteration. Results sort by normalized title, original primary title, year, and IMDb ID; exact ties retain source order. Duplicate terminal lists use stable sorting, and different keys emerge through lexicographic traversal.

Additional algorithms and containers include binary search of Trie child vectors, iterative DFS stacks, vectors of IDs, a sorted-scan correctness oracle, steady-clock timing, and fixed-seed mt19937 workload generation. A set removes duplicate benchmark queries; it is not the search index. CMake builds standard C++17 sources, and CTest runs dependency-free test executables. The application needs no web service, database, external C++ framework, or paid API.

Responsibilities

Aidan Quach: Phase 1 architecture, CMake, shared types/interface, normalization, streaming loader, manual Trie, initial CLI, fixtures/tests, setup documentation, and handoff. Phase 1 was merged through PR #5.

Hana Elbatouty: TST node structure, insertion, prefix search, and TST tests, including branching, ordering, and result limits.

Krish Gupta: result-equivalence tests, deterministic workload, benchmark engine, final terminal menu/commands, integration and real-data tests, and measured analysis documentation. Phase 2 was merged through PR #11.

Worst case complexity

Let B be source bytes scanned, W the longest source line, M the bytes of selected movie records, C the total normalized title length, V the number of index nodes, E the new Trie edges, A the alphabet size (at most 37), P the raw prefix length, L its normalized length, S the subtree nodes visited, and R the IDs returned. Let D = sum(k (log k)^2) over terminal multiplicities k, a conservative stable-sort comparison bound [5]. String comparisons additionally depend on string length.

Major function	Worst case time or total build bound
Normalize a title or query	O(input bytes), with O(input bytes) output capacity
Stream and filter the dataset	O(B), excluding fixed-size header setup; O(M + W) retained memory
Build the sparse Trie	O(C log A + E A + D), including sorted child-vector shifts
Build the unbalanced TST	O(C A + D), including worst-case sibling chains
Trie prefix search	O(P + L log A + S + R)
TST prefix search	O(P + L A + S + R)
Iterate index statistics	O(V); capacities are summed outside query timings

Vector growth gives amortized constant-time node append over a build. One insertion can additionally pay O(V) for node-vector reallocation; sorted child insertion can shift O(A) links. The fixed alphabet bounds these factors, but showing them explains different work. Stable sorting uses O(k log k) comparisons with enough auxiliary memory, or O(k (log k)^2) otherwise [5]. Thus oracle sorting needs O(n (log n)^2) comparisons in the fallback worst case, normally O(n log n).

Each index occupies O(V + n) owned storage, including terminal IDs; V is bounded by C plus the Trie's root. Query scratch storage includes the normalized prefix, returned IDs, and an explicit traversal stack proportional to traversal depth. The ten-result limit bounds R, not all work: a long matching title can still require many node visits. In the worst case an exhaustive search visits the entire matching subtree.

Proposal comparison and design rationale

The original proposal called for autocomplete as a user types, two manually implemented structures, 150,000 IMDb movies, and comparisons across at least 100 prefixes [6]. The completed program keeps the proposed structures, dataset size, ordering requirement, and measurements. It accepts a prefix after the user presses Enter rather than updating suggestions on each keystroke; the terminal menu provides a simple interface for running and comparing searches.

The implementation also specifies details the proposal left open: a sparse Trie to limit child-link storage, consistent ASCII normalization and duplicate ordering, an independent sorted scan to check results, and CSV export for benchmark data. The proposal assigned data preparation to Krish; Aidan implemented the loader in Phase 1, while Krish added real-dataset validation. These choices kept the two structures on the same records and made their results easier to compare.

Experiment and measured results

The published experiment used the November 4 snapshot, 150,000 movies, an Intel Core Ultra 9 185H laptop with 32 GB RAM, Windows 11, GCC 13.1, and a Release build [3]. The workload has 130 unique prefixes: short, medium, long, common, rare, and verified no-match categories. Five fresh builds alternate structure order. Three untimed warm-up passes precede 50 measured passes at limit ten, plus five exhaustive passes. Query timing surrounds only search calls; loading, display, validation, and result destruction are outside that interval.

Metric from one documented Windows run	Trie	TST
Median construction time, milliseconds	366.4	229.2
Nodes allocated	1,395,441	1,395,440
Estimated index storage, MiB	119.00	113.17
Median query time at limit ten, microseconds	3.700	2.600
Mean query time at limit ten, microseconds	3.940	2.797
Mean nodes visited at limit ten	66.9	75.8

All 7,150 cross-structure result-list comparisons agreed; both indexes also agreed with the independent sorted scan. The TST built faster, used about 4.9% less estimated storage, and answered bounded queries faster in this run. Both represent the same nonempty prefixes; the Trie's extra root accounts for the one-node difference. Memory differences therefore arise from node/link layout and allocated capacity, rather than a large difference in node counts.

The TST visited more nodes while taking less time. Node visits are not equal units of work: a Trie visit can include several comparisons and a lookup in a separate child-vector allocation, while a TST visit examines one character node. Better locality is a plausible explanation, not a measured causal result; cache counters were not collected. These results compare these implementations, not all possible Tries and TSTs. A fixed-array Trie would have a different memory/speed trade-off.

Validation and limitations

The test suite covers normalization, parsing/filtering, duplicate ordering, long keys, rebuilds, randomized oracle comparisons, CLI/menu behavior, and deliberately incorrect benchmark results. On November 5, all six CTest targets passed on macOS in Release and under AddressSanitizer/UndefinedBehaviorSanitizer, including 150,000 official records. A separate current-snapshot benchmark also produced 130 queries and zero mismatches across 7,150 comparisons. These checks supplement the Windows/Linux validation in [3]; they are not the source of the table above.

Limitations include ASCII-only matching, an older-film selection, no popularity ranking, an unbalanced TST, and timings affected by machine load and clock resolution. Estimated storage counts allocated node/child/terminal vector capacities, not process RAM, allocator overhead, shared movie records, or query scratch space. Snapshot and toolchain differences can change the measurements.

Group reflection

Draft status: this section needs the team's actual experience before submission. The technical record supports the discussion prompts below, but it cannot establish what each person felt or learned. Replace these prompts with a coherent group reflection; keep the final report within five pages at 12-point type.

Overall experience: Team response pending. Describe how the two-stage implementation and handoff worked, whether responsibilities were clear, and how effectively the group coordinated. Include both positive aspects and anything that did not work as expected.

Challenges: Team response pending. Confirm the difficulties actually encountered. Code-supported topics include preserving movie IDs during vector reallocation, traversing TST endpoint siblings correctly, ordering duplicate-title records identically, validating a large TSV without loading it all, and obtaining repeatable timings across toolchains. Explain how the team solved the challenges it really experienced.

Changes if starting again: Team response pending. Possible discussion topics are agreeing on interfaces and canonical ordering earlier, setting up shared correctness tests before benchmarking, pinning the dataset snapshot, reserving time for the report/video, and comparing memory-layout alternatives. State the team's own decisions rather than treating these suggestions as past experience.

Individual learning: Each member's response is pending. Aidan, Hana, and Krish should each describe a specific lesson from their work. One specific, truthful takeaway per member is better than a generic claim that everyone learned the same thing.

References

[1] Group 123. Movie Title Autocomplete source code, README, and merged implementation history. Reviewed main commit 13148ef. https://github.com/aidanq06/movie-title-autocomplete

[2] IMDb. Non-Commercial Datasets documentation and title.basics.tsv.gz. https://developer.imdb.com/non-commercial-datasets/ ; https://datasets.imdbws.com/title.basics.tsv.gz . Documentation accessed November 5, 2026. Information courtesy of IMDb (https://www.imdb.com). Used with permission; applicable IMDb terms govern the data.

[3] Group 123. Analysis notes, dataset provenance, benchmark methodology, and recorded measurements. https://github.com/aidanq06/movie-title-autocomplete/blob/main/docs/ANALYSIS_NOTES.md

[4] COP3530. Project 2b Group Submission rubric, Fall 2026. Assignment text supplied by the team; report and video limits follow that text.

[5] Microsoft Learn. C++ Standard Library algorithm functions, stable_sort complexity. https://learn.microsoft.com/en-us/cpp/standard-library/algorithm-functions#stable_sort . Accessed November 5, 2026.

[6] Movie Search, Group 123. Movie Title Autocomplete - Project 2 Proposal. October 2026.